

04 - Security, APIs & Third-Party Decisions

What to use, what not to use, and how to stay secure without over-building

GROW MY THERAPY - STAGE 2 SELECTION ASSIGNMENT

1. Important scope decision

DO NOT OVERBUILD The assignment does not require a backend, login, database, payments, AI API, or admin dashboard. A static frontend is the safest submission because it lets you spend time on the areas they actually score.

2. Recommended third-party stack

Package/service	Use	Required?
Next.js	Framework and App Router rendering	Required by assignment
Tailwind CSS	Responsive styling and reusable design tokens	Required by assignment
next/image	Image optimization and layout stability	Built in / recommended
next/font	Efficient font loading	Built in / recommended
Lucide React	A few clean interface icons if needed	Optional
clsx + tailwind-merge	Safe conditional class composition	Optional
Vercel	Deployment and preview links	Recommended
Playwright	Critical end-to-end checks	Optional but professional
Vercel Analytics	Only if you want basic anonymous performance/traffic insight	Optional; not needed for selection

3. Image sources

- Maya portrait: use the supplied profile image.
- Office section: use the supplied office images exactly as requested by the assignment.

- Other homepage images: use properly licensed stock imagery from a reputable source or original/AI-generated imagery with consistent visual direction. Track the source/license in README.
- Do not hotlink third-party images in production; download/optimize permitted assets and store them locally in public/images.
- Do not reuse the Conejo Valley site photography in the redesign because Part 2 specifically asks for all images to be replaced.

4. If you add a contact form - secure minimal backend

Only add a form if it materially improves the demo and you have enough time. For a mental-health site, do not invite users to enter diagnosis details, trauma history, medication information, emergency information, or other sensitive health content into a generic web form.

1. Use a Next.js Server Action or Route Handler.
2. Validate on the server with Zod (name, email, short general message only).
3. Add a honeypot and/or Cloudflare Turnstile for bot protection.
4. Rate-limit requests by a privacy-conscious identifier. Upstash Redis can be used, but it is unnecessary for a purely static assignment.
5. Send form data to a transactional provider such as Resend only if configured. Keep API keys in environment variables and never expose them with NEXT_PUBLIC_.
6. Return generic errors; never echo secrets or stack traces to the browser.
7. Do not log the full submitted message. Log only event IDs/status needed for debugging.
8. Add a clear disclaimer that the form is not for emergencies and should not contain sensitive clinical information if you choose to include it.

5. Security headers

Recommended production headers:

- Content-Security-Policy (tighten to actual image/font/script origins)
- Referrer-Policy: strict-origin-when-cross-origin
- X-Content-Type-Options: nosniff
- Permissions-Policy: camera=(), microphone=(), geolocation=()
- Strict-Transport-Security on production HTTPS
- frame-ancestors 'none' or an equivalent X-Frame-Options policy when appropriate

6. Environment-variable rules

- Commit .env.example with variable names only.
- Never commit .env.local, provider API keys, tokens, or private URLs.
- Only variables intentionally safe for browsers should use the NEXT_PUBLIC_ prefix.
- Rotate any key immediately if it is accidentally committed; deleting a commit does not reliably make a leaked secret safe.

7. SEO + privacy for a fictional therapist

- Implement real metadata, semantic headings, Open Graph fields and local-service copy because the assignment explicitly evaluates SEO-aware copy.
- Set the deployed demo to noindex so search engines do not index a fictional clinician/address as a real service.
- Do not add fabricated testimonials or review schema.

- Do not use MedicalOrganization/Physician schema unless you are certain it is semantically and legally appropriate for a real deployment; for this fictional demo, schema is not necessary.

8. Dependency minimization

A strong submission can be built with Next.js, Tailwind, one icon package, and testing tools. Avoid installing Redux, React Query, database clients, authentication, form libraries, animation frameworks, chart libraries, CMSs, or AI SDKs unless the design genuinely requires them. Fewer dependencies mean a smaller attack surface and fewer reasons the demo can break.

9. README dependency disclosure

Include a short “Third-party tools and assets” section in README that names the deployment platform, packages, font source, stock image source/license, and any optional contact service. This is enough to demonstrate professional transparency without burying the evaluator in infrastructure details.